

Теория формальных языков

Лабораторная работа №4

Вариант 8

Работу выполнила:

Студент группы ИУ9-51Б

Григорьева Софья

Содержание

1	Задание	2
2	Анализ на КС свойство	4
3	Наивный парсер	5
4	Оптимизированный парсер	5
5	Тестирование	6
6	Сравнение парсеров	7

1 Задание

Дан язык:

$$^((? : a \mid b)^*) a ((? : a \mid b)^*) \setminus 1 b \setminus 2 (\setminus 2)^+ \$$$

1. Проанализировать язык на наличие свойства контекстно-свободности (КС-свойство). В случае его наличия — проанализировать язык на регулярность.
2. Построить “наивный” парсер слов для языка, используя рекурсивный разбор с возвратами. Парсер не должен заикливаться.
3. Построить оптимизированный парсер слов для языка. Оценить сверху его вычислительную сложность.
4. Посредством фазз-тестирования проверить эквивалентность парсеров и построить сравнительные графики скорости их работы:
 - на случайных словах, принадлежащих языку;
 - на случайных словах, не принадлежащих языку.

Используются два тестовых пула.

Требования к оформлению

Пункт 1 допускается выполнять вручную. Для фиксации решения рекомендуется использовать формат `Markdown` со встроенными формулами `LaTeX`, поддерживаемыми `MathJax`, либо формат `Markdown Obsidian` (он допускает расширенную поддержку формул `LaTeX`). Допускается сдача фото рукописного решения, однако за оформление в `Markdown/LaTeX` добавляется 1 балл.

Расширенные регулярные выражения

В расширенных регулярных выражениях используется нестандартная семантика ссылок на группы захвата.

Ссылка на группу захвата может встречаться текстуально *раньше*, чем сама группа захвата (вплоть до появления до её начала). Это означает, что при сопоставлении с таким регулярным выражением первый проход распознавателя осуществляется по альтернативе, в которой выражение инициализируется (то есть ссылок на него ещё нет).

Пример. Рассмотрим регулярное выражение:

$$(\backslash 2a \mid (\hat{b}\backslash 1) \mid bb)^* \$$$

Здесь:

- вторая группа ссылается на первую;
- первая группа может быть инициализирована только по третьей альтернативе под итерацией;
- только после инициализации второй группы на второй альтернативе под итерацией может быть осуществлён проход по первой альтернативе.

Маркеры начала и конца строки

Все регулярные выражения оснащены маркерами начала строки $\wedge$ и конца строки $\$$. Это сделано для того, чтобы:

- иметь возможность определять опережающие проверки до самого конца строки (если выражение внутри опережающей проверки заканчивается маркером $\$$);
- либо проверять только префикс оставшейся строки.

Маркеры начала и конца строки не входят в алфавит языка.

2 Анализ на КС свойство

Язык не является КС.

Доказательство:

Воспользуемся леммой о накачке.

Лемма о накачке (для КС-языков). Существует такое число $p > 0$, что любое слово $s \in L$ длины не меньше p можно разложить как

$$s = uxvuw$$

так, чтобы выполнялись условия:

1. $|xvy| \leq p$,
2. $|xy| > 0$,
3. $ux^i v y^i w \in L$ для всех $i \geq 0$.

Рассмотрим слово

$$s = a^p a b^p a^p b b^p b^p \in L.$$

Разобьём его на блоки и пронумеруем их://

$$\underbrace{a^p}_{\text{блок 1}} \underbrace{a}_{\text{блок 2}} \underbrace{b^p}_{\text{блок 3}} \underbrace{a^p}_{\text{блок 4}} \underbrace{b}_{\text{блок 5}} \underbrace{b^p}_{\text{блок 6}} \underbrace{b^p}_{\text{блок 7}}$$

Из-за условия $|xvy| \leq p$ возможна накачка только следующих блоков:

$$1, 2, 3, 4, 5, 6, 7, \quad 1-2, 1-3, 2-3, 3-4, 4-5, 4-6, 5-6, 6-7$$

Однако, из-за групп захвата и структуры слова:

- Блок 1 (a^p) не может быть накачан отдельно без блока 4 (a^p).
- Блок 3 (b^p) не может быть накачан без блоков 6 и 7 ($b^p b^p$).
- Блоки 2 и 5 не могут быть накачаны так как невозможно перераспределение в блоки 1, 3, 4 и 6.

Следовательно, условия леммы не выполняются и язык не КС.

3 Наивный парсер

Исходный код представлен в файле `parsers.ts` и инициализируется с помощью функции `buildParser`.

Парсер реализован рекурсивно и перебирает все возможные варианты разбора строки.

4 Оптимизированный парсер

Исходный код представлен в функции `optimalParser` в файле `parsers.ts`.

Оптимизированный парсер использует следующую стратегию: сначала подбираются все возможные значения первой группы, затем для каждого значения первой группы подбирается значение второй группы с учётом того, что после первой группы следует символ `a`, а после второй — символ `b`.

Для оптимизации применяется следующее: прямое индексирование и срезы, свойства языка: первая группа повторяется дважды, вторая — не менее трёх раз; в словах обязательно присутствуют хотя бы одна буква `a` и одна буква `b` (длина слова не меньше 2 символов), а также работа с длинами, заключающаяся в следующем:

1. Оценка максимально возможной длины первой группы.
2. Оценка максимально возможной длины второй группы.
3. Проверка корректности длины второй группы в зависимости от длины первой.
4. Проверка возможности повторений второй группы в конце слова (кроме того вместо перебора всех возможных разбиений на конце используется сравнение с заранее известным концом, поскольку группа 2 известна и количество повторений можно определить по длине).

Сложность работы парсера в худшем случае составляет $O(n^2)$.

5 Тестирование

Исходный код тестирования представлен в функции `testParsers` в файле `parsers.ts`.

Тестирование включает:

- Генерацию слов, принадлежащих языку, и слов, не принадлежащих языку.
- Прогон обоих парсеров на каждом слове.
- Сравнение результатов работы парсеров.

6 Сравнение парсеров

На графиках представлено сравнение производительности парсеров для двух типов слов: принадлежащих языку и не принадлежащих ему.

